20261. IT6390E – Semantic Web

Assignment 5.9. RDFS

Student ID: 20261059M – Class: CH2026A

Question. Using the base graph of Part 2, Ex. 2, explain whether the following statements are *true* (i.e. RDFS-entailed by the graph) or *unknown*:

1. `:Ann :isChildOf :Carl .`
2. `:Ann :hasParent :Carl .`
3. `:Ann :hasParent _:x .`
4. `:Ann :hasParent [ rdf:type :Person ] .`

1. The base graph

Schema part (terminological knowledge).

```
:Man        rdfs:subClassOf   :Person .
:Parent     rdfs:subClassOf   :Person .
:Child      rdfs:subClassOf   :Person .
:Father     rdfs:subClassOf   :Parent ;   rdfs:subClassOf :Man .

:hasParent  rdfs:domain       :Person ;   rdfs:range      :Parent .
:hasFather  rdfs:subPropertyOf :hasParent ; rdfs:range    :Father .
:isChildOf  rdfs:domain       :Child ;    rdfs:range      :Parent .
```

Data part (assertional knowledge).

```
:Ann        rdf:type          :Person ;   :hasFather      :Carl .
:Carl       rdf:type          :Man .
```

2. RDFS entailment rules used

Rule	Premises		Conclusion
rdfs2	`?p rdfs:domain ?C .  ?x ?p ?y .`	⇒	`?x rdf:type ?C .`
rdfs3	`?p rdfs:range ?C .  ?x ?p ?y .`	⇒	`?y rdf:type ?C .`
rdfs7	`?p rdfs:subPropertyOf ?q .  ?x ?p ?y .`	⇒	`?x ?q ?y .`
rdfs9	`?C rdfs:subClassOf ?D .  ?x rdf:type ?C .`	⇒	`?x rdf:type ?D .`
rdfs11	`?C rdfs:subClassOf ?D .  ?D rdfs:subClassOf ?E .`	⇒	`?C rdfs:subClassOf ?E .`
se2	`?x ?p ?y .`	⇒	`?x ?p _:b .`

Rule se2 is the generalisation rule of simple entailment: an IRI in object position may be replaced by a blank node.

Two general principles are also needed. First, a blank node is an existential variable: a graph containing `_:b` is entailed as soon as *some* resource can be substituted for `_:b` so that all triples become entailed. Second, RDFS follows the open-world assumption and has no negation, so a statement that cannot be derived is not thereby false; it is simply unknown.

3. Facts derived from the base graph

Starting from `:Ann :hasFather :Carl` and `:Carl rdf:type :Man`, the RDFS closure of the data part contains:

(D1) `:Ann :hasParent :Carl .`
rdfs7, from `:hasFather rdfs:subPropertyOf :hasParent` and `:Ann :hasFather :Carl`.

(D2) `:Carl rdf:type :Father .`
rdfs3, from `:hasFather rdfs:range :Father` and `:Ann :hasFather :Carl`.

(D3) `:Carl rdf:type :Parent .`
rdfs9, from (D2) and `:Father rdfs:subClassOf :Parent`. It also follows from (D1) by rdfs3 with `:hasParent rdfs:range :Parent`.

(D4) `:Carl rdf:type :Person .`
rdfs9, from `:Carl rdf:type :Man` and `:Man rdfs:subClassOf :Person`. Equivalently, `:Father rdfs:subClassOf :Person` by rdfs11 applied to (D2).

(D5) `:Ann rdf:type :Person .`
rdfs2, from `:hasParent rdfs:domain :Person` and (D1); it is also asserted explicitly.

No rule of the RDFS closure produces a triple whose predicate is `:isChildOf`, and nothing in the closure gives `:Ann rdf:type :Child`. This observation is what separates statement 1 from the other three.

4. Answers

Statement 1: `:Ann :isChildOf :Carl .` — **UNKNOWN**

In the base graph `:isChildOf` appears only in schema triples: it is typed as an `rdf:Property` and given a domain (`:Child`) and a range (`:Parent`). It is never used in an assertion, and it is not linked to `:hasParent` or `:hasFather` by `rdfs:subPropertyOf`.

The RDFS rules can only move a triple from a property to one of its super-properties (rdfs7) or infer a type from a domain or range declaration (rdfs2, rdfs3). None of them can create a new triple whose predicate is `:isChildOf`. In particular, RDFS has no construct for inverse properties (`owl:inverseOf` belongs to OWL, not to RDFS), so the intuitive reading that "`:isChildOf` is the inverse of `:hasParent`" is nowhere expressed in the graph; it exists only in the names chosen by the modeller, which carry no formal meaning.

Therefore the statement is not entailed. Since RDFS cannot express negation and reasons under the open-world assumption, the statement is not false either: adding it to the graph would keep the graph consistent, and would additionally entail `:Ann rdf:type :Child` through the domain of `:isChildOf`. Its truth value is thus *unknown*.

Statement 2: `:Ann :hasParent :Carl .` — **TRUE**

The graph asserts `:Ann :hasFather :Carl`, and the schema declares `:hasFather rdfs:subPropertyOf :hasParent`. Rule rdfs7 gives directly

`:hasFather rdfs:subPropertyOf :hasParent .` `:Ann :hasFather :Carl .`
$\Rightarrow$ `:Ann :hasParent :Carl .`

This is exactly (D1), so the statement is RDFS-entailed and therefore true. Intuitively: whoever is Ann's father is in particular one of her parents.

Statement 3: `:Ann :hasParent _:x .` — TRUE

The blank node `_:x` is an existential variable, so the statement reads "Ann has at least one parent" without saying who it is. To prove it, it is enough to exhibit one resource that can be substituted for `_:x` so that the resulting triple is entailed.

Take the mapping `_:x` ↦ `:Carl`. The instance obtained is `:Ann :hasParent :Carl`, which is (D1) and is entailed by statement 2. By generalisation (rule se2, an IRI in object position may be replaced by a blank node), the graph entails `:Ann :hasParent _:x`.

A statement with a blank node is always weaker than the corresponding statement with a named resource, so the entailment of statement 2 makes statement 3 immediate.

Statement 4: `:Ann :hasParent [ rdf:type :Person ] .` — TRUE

The square brackets are Turtle syntax for an anonymous resource, so the statement expands into a two-triple graph sharing one blank node:

```
:Ann :hasParent _:b .     _:b rdf:type :Person .
```

It reads "Ann has a parent who is a person". Both triples must be satisfied by the *same* substitution of `_:b`. Take again `_:b` ↦ `:Carl`:

- `:Ann :hasParent :Carl` is (D1), obtained by rdfs7;
- `:Carl rdf:type :Person` is (D4), obtained by rdfs9 from `:Carl rdf:type :Man` and `:Man rdfs:subClassOf :Person`.

Both instantiated triples belong to the RDFS closure of the graph, hence the whole two-triple graph is RDFS-entailed and the statement is true. The same conclusion can be reached without using `:Carl rdf:type :Man`: the range of `:hasFather` gives `:Carl rdf:type :Father` (D2), and `:Father` is a subclass of `:Parent`, which is a subclass of `:Person`.

5. Summary

No.	Statement	Value	Justification
1	`:Ann :isChildOf :Carl .`	Unknown	No rule derives an `:isChildOf` assertion; RDFS has no inverse properties.
2	`:Ann :hasParent :Carl .`	True	rdfs7 on `:hasFather rdfs:subPropertyOf :hasParent`.
3	`:Ann :hasParent _:x .`	True	Weaker than statement 2; take `_:x` ↦ `:Carl`.
4	`:Ann :hasParent [ rdf:type :Person ] .`	True	`_:b` ↦ `:Carl`, with `:Carl rdf:type :Person` by rdfs9.